Pacing and Congestion-Window Growth: An Exact Packet-Level Analysis

Skai Uijing

October 8, 2026

Abstract

An unchanged per-ACK window update can produce different growth in physical time when newly available window credit passes through a pacer. We study this effect in a loss-free, fixed-delay packet model: each slow-start ACK reserves two packets, their spacing is set by the updated window and a fixed gain, and existing reservations retain their timestamps. We solve the transmission recurrence exactly for every gain $g \geq 1$. A fixed-batch comparator supplies a Fibonacci lower bound, while immediate transmission supplies the doubling upper bound. The exact solution identifies a transition at $g = 2\ln 2$: below it, additional ACK waiting ends after a finite prefix; above it, each RTT contains a sending interval followed by persistent waiting. The sending span approaches $(2\ln 2/g)R$, and the normalized window growth exponent is $\min\{g/2, \ln 2\}$. Packet timestamps converge to those of the immediate-send model as gain tends to infinity. For loss-free congestion avoidance, we derive an exact event recurrence and a queue-clock decomposition with a logarithmic term, leaving the cumulative ACK-waiting contribution explicit. All results concern the specified scheduler; no equivalence to a deployed TCP stack or network-performance improvement is assumed.

1 The timing question

This work began while developing SCP, a TCP-like transport with a Reno-like controller. Replacing burst sending with pacing made transfers slower in the author's development runs, despite little change in the useful-data fraction. That observation motivated a narrower mathematical question: with the window update held fixed, how does transmission scheduling change the times at which ACKs return and the window grows? The observation is motivation, rather than evidence that the model below explains the full implementation's performance.

Consider a one-packet initial window. Send that packet at time zero. Its ACK returns at R and raises the window to two packets. If the next two packets leave at R and $1.5R$, only the first has been acknowledged at $2R$: the window is three, not four. The window was updated immediately at each ACK. The difference comes from transmission times, not from postponing window updates to RTT boundaries.

The analysis follows three steps. We first specify a packet scheduler and separate actual flight from future reservations. We then compare its growth with two solvable extremes, before obtaining its exact transmission times for general pacing gain. Finally, we carry the same event accounting into congestion avoidance. The main object is $\mathrm{cwnd}(t)$ and its underlying packet trace, not an optimization of network throughput or a replacement congestion controller.

2 Model and event accounting

2.1 Assumptions and units

Every packet has payload size M, and its ACK returns exactly $R > 0$ after actual transmission. There is no loss, reordering, ACK aggregation, application limitation, receiver-window limitation,

or link-serialization constraint. These assumptions isolate feedback timing. In particular, an instantaneous burst is an admissible comparison trace in this ideal model.

Write $\tau = t/R$ and $w(\tau) = \mathrm{cwnd}(R\tau)/M$. All timestamps below are in units of R, and windows and packet counts are in units of M. The initial window is one packet; packet 0 is sent at $s_0 = 0$. During the slow-start analysis, the connection remains in slow start indefinitely and every ACK increases w by one. This is the full-packet, one-ACK-per-packet specialization of the slow-start update; Section 8 treats a finite threshold and the real-valued congestion-avoidance update [1].

At a shared timestamp, process ACKs before transmissions and sample after all events there. Sampling at integer τ does not change the event schedule. The gain $g \geq 1$ is fixed. Notation used throughout is collected in Table 1.

Table 1: Notation. Physical timestamps equal normalized timestamps times R.

Symbol	Meaning
F, Q	Sent but unacknowledged packets; reserved but unsent packets
s_j	Actual send time of packet j, numbered from zero
a_k, w_k	Arrival time of ACK k, and window immediately after it
b_k, T_k	First send in ACK k's batch; next free slot after that batch
$N_g(\tau)$	Number of actual transmissions by τ, including packet 0
H_k	$\sum_{r=1}^{k} 1/r$, with $H_0 = 0$
$D(j)$	$H_{2j} - H_j$; a harmonic sum, not a traffic counter
B_m	b_{2^m}, the first send associated with an ACK-index block
f_n	Fibonacci numbers, $f_0 = 0$, $f_1 = 1$

2.2 Window credit and the append rule

After each reservation step, the scheduler maintains

$$F + Q = w. \tag{2.1}$$

An actual transmission transfers one packet from Q to F, preserving the sum. An ACK reduces F by one and increases w by one, creating exactly two new reservation positions. Reserving two packets restores (2.1). Hence $F \leq w$ at every transmission, and queued packets cannot consume the same credit twice.

After ACK k, $w_k = k+1$. The two new packets are appended after every existing reservation, with spacing $1/[g(k+1)]$. Existing timestamps remain unchanged. The scheduler also retains one spacing after the last packet of each batch. With $T_0 = 1/g$, its complete recurrence is

$$a_k = s_{k-1} + 1, \qquad b_k = \max\{a_k, T_{k-1}\}, \tag{2.2}$$

$$s_{2k-1} = b_k, \qquad s_{2k} = b_k + \frac{1}{g(k+1)}, \tag{2.3}$$

$$T_k = b_k + \frac{2}{g(k+1)}. \tag{2.4}$$

Here T is the next free slot after the *entire* queue, even when that queue is long. The rate is selected at reservation time. It is not recalculated at the eventual transmission of an already reserved packet. This distinction is part of the model, not a consequence of specifying $g\mathrm{cwnd}/R$ alone.

Packet sends are strictly ordered for finite g. The fixed feedback delay therefore preserves their order, justifying $a_k = s_{k-1} + 1$. Each reservation is causal because $b_k \geq a_k$. Also, $b_1 = 1$, and for $k \geq 2$,

$$b_k = \max\left\{a_k, b_{k-1} + \frac{2}{gk}\right\}. \tag{2.5}$$

In particular, the ACK branch can be written without packet timestamps:

$$a_{2j} = b_j + 1, \qquad a_{2j+1} = b_j + 1 + \frac{1}{g(j+1)} \quad (j \geq 1). \tag{2.6}$$

2.3 From actual transmissions to the window

Equation (2.5) implies $b_k \geq 1 + (2/g)(H_k - 1) \to \infty$, so only finitely many packets leave in any finite time interval. Set $N_g(x) = 0$ for $x < 0$. For $x \geq 0$, count actual sends, not reservations:

$$N_g(x) = 1 + \sum_{k \geq 1} \left(\mathbf{1}_{\{b_k \leq x\}} + \mathbf{1}_{\{b_k + 1/[g(k+1)] \leq x\}} \right). \tag{2.7}$$

The slow-start window is exactly

$$\boxed{w_g(\tau) = 1 + N_g(\tau - 1).} \tag{2.8}$$

Gain changes the times of ACKs, not the window $k + 1$ after ACK k.

For integer samples, write $W_n = w_g(n)$ and let S_n count sends in $(n, n+1]$. Those packets are acknowledged in $(n+1, n+2]$, so

$$\boxed{W_{n+2} = W_{n+1} + S_n.} \tag{2.9}$$

This identity is shared by the comparison models below. It neither assumes RTT-batched ACK processing nor determines S_n without a scheduling rule.

3 Two comparison models

3.1 Immediate transmission gives the upper bound

For any sender satisfying these slow-start assumptions and the flight constraint, the ACKs in $(n, n+1]$ can only come from packets in flight just after n. A packet sent strictly after n is acknowledged strictly after $n + 1$. Each returning ACK adds one window unit, hence

$$W_{n+1} - W_n \leq F(n) \leq W_n. \tag{3.1}$$

Since $W_0 = 1$, induction gives $W_n \leq 2^n$. The sender that immediately sends both newly permitted packets at every ACK attains this bound. Its physical window at the boundaries is

$$\mathrm{cwnd}_{\mathrm{burst}}(nR) = M 2^n. \tag{3.2}$$

This is a timing bound in a fixed-delay, unlimited-service model, not a claim that burst sending maximizes throughput on a congested network.

3.2 A fixed-batch comparator gives Fibonacci growth

Define a separate sender that reads its window $\widehat{W}_n$ at each integer time n after ACK processing, and sends $\widehat{W}_n$ packets at

$$n + \frac{i}{\widehat{W}_n}, \qquad i = 0, \ldots, \widehat{W}_n - 1. \tag{3.3}$$

ACKs still increase its window immediately, but do not change the current batch. In $(n, n+1]$ there are $\widehat{W}_n - 1$ remaining packets of this batch and the first packet of the next batch. Equation (2.9) gives

$$\widehat{W}_{n+2} = \widehat{W}_{n+1} + \widehat{W}_n, \quad \widehat{W}_0 = 1, \quad \widehat{W}_1 = 2, \qquad \widehat{W}_n = f_{n+2}. \tag{3.4}$$

Its window sequence is $1, 2, 3, 5, 8, 13, \ldots$. Its explicit boundary formula is

$$\mathrm{cwnd}_{\mathrm{fixed}}(nR) = \frac{M}{\sqrt{5}}(\phi^{n+2} - \psi^{n+2}), \qquad \phi = \frac{1+\sqrt{5}}{2}, \quad \psi = \frac{1-\sqrt{5}}{2}. \tag{3.5}$$

The sender respects the actual-flight limit. Indeed, in a batch with previous size $p = f_{n+1}$ and current size $q = f_{n+2}$, its flight just after phase $0 \le u < 1$ is $p + \lfloor qu \rfloor - \lfloor pu \rfloor \le q$, while its window is at least q. Unlike the append scheduler, it permits future plans to rely on future flight releases; it does not enforce the same reservation invariant $F + Q = w$ at planning time.

Solving this comparator alone does not make it a lower bound. The needed comparison is between *actual* send times.

Theorem 3.1 (Fibonacci-to-doubling sandwich)**.** *For the append scheduler of Section 2, every fixed $g \ge 1$ satisfies, for integers $n \ge 0$,*

$$\boxed{M f_{n+2} \le \mathrm{cwnd}_g(nR) \le M 2^n.} \tag{3.6}$$

The upper bound was proved above. Appendix A proves that at $g = 1$ each packet leaves no later than its fixed-batch counterpart. Fixed feedback delay then preserves the ordering of ACKs and windows. The gain monotonicity proved in Section 7 extends the lower bound to $g \ge 1$. The comparison is not justified merely by reserving packets earlier, and it does not assert the local service condition $S_n \ge W_n$ for arbitrary pacers.

4 Exact transmission times for every gain

Let $c = 2/g$ and $D(j) = H_{2j} - H_j$. Two elementary facts will identify which branch of (2.5) is active:

$$D(j+1) - D(j) = \frac{1}{(2j+1)(2j+2)} > 0, \qquad D(j) < \ln 2, \qquad D(j) \longrightarrow \ln 2. \tag{4.1}$$

The inequality follows by comparing the sum with $\int_j^{2j} dx/x$; the limit follows from $H_j = \ln j + \gamma_E + O(1/j)$, where γ_E is Euler's constant.

Group ACK indices as $q \le k < 2q$, where $q = 2^m$, and write $B_m = b_{2^m}$. These groups are analytical partitions of the event indices. The sender does not wait for a group or an RTT boundary to process an ACK.

Theorem 4.1 (Unified exact solution)**.** *For every fixed $g \ge 1$,*

$$B_0 = 1, \qquad B_{m+1} = B_m + \max\{1, cD(2^m)\}, \tag{4.2}$$

$$b_k = B_m + c(H_k - H_{2^m}), \qquad 2^m \le k < 2^{m+1}. \tag{4.3}$$

Equivalently, with $[x]_+ = \max(x, 0)$ and an empty sum equal to zero,

$$\boxed{b_k = cH_k + 1 - c + \sum_{i=0}^{\lfloor \log_2 k \rfloor - 1} [1 - cD(2^i)]_+.} \tag{4.4}$$

Together with (2.3), this determines every packet timestamp.

Proof. The first group contains only $k = 1$, for which $b_1 = 1$. Assume the formulas hold before a group with $q = 2^m$, $m \ge 1$. At its first index,

$$a_q = b_{q/2} + 1 = B_{m-1} + 1,$$
$$T_{q-1} = b_{q-1} + c/q = B_{m-1} + c(H_q - H_{q/2}).$$

Their maximum proves the formula for B_m. Set $\Delta = B_m - B_{m-1} \geq 1$.

Within this group, proceed in increasing k. For an even $k = 2j > q$, the already established formulas give

$$T_{2j-1} - a_{2j} = \Delta - 1 + c[D(j) - D(q/2)] > 0. \tag{4.5}$$

Here $q/2 < j < q$, and D strictly increases. For an odd $k = 2j + 1 > q$,

$$T_{2j} - a_{2j+1} = \Delta - 1 + c\left[D(j) - D(q/2) + \frac{1}{2j+1} - \frac{1}{2j+2}\right] > 0. \tag{4.6}$$

The last difference is positive even when $j = q/2$. Thus every interior step selects the queue-tail branch of (2.5). Summing c/k from the group start proves (4.3) and completes strong induction.

Finally expand (4.2), use $\max(1, x) = x + [1 - x]_+$ and telescope $\sum_{i<m} D(2^i) = H_{2^m} - 1$ to obtain (4.4). □

The proof also identifies all possible extra waiting: apart from the initial ACK, it can occur only at ACK indices 2^m. This conclusion follows from the two branches of the actual recurrence, rather than an assumption that the pacer stays busy.

5 Sending spans, waiting, and the gain transition

5.1 Finite-group formulas

Group m, with $q = 2^m$, consists of q two-packet batches. Its $2q$ packets have indices $2q - 1, \ldots, 4q - 2$. The first leaves at B_m; the last leaves at

$$s_{4q-2} = b_{2q-1} + \frac{1}{2gq} = B_m + \frac{2}{g}D(q) - \frac{1}{2gq}. \tag{5.1}$$

Let L_m be the physical time from that first send to that last send. Let I_m be the extra wait between the final reserved slot of this group and the start of the next, and let G_m be the actual last-to-next-first gap. Then

$$\boxed{\begin{aligned} L_m/R &= \frac{2}{g}D(q) - \frac{1}{2gq}, \\ I_m/R &= \left[1 - \frac{2}{g}D(q)\right]_+, \\ G_m/R &= I_m/R + \frac{1}{2gq}. \end{aligned}} \tag{5.2}$$

The first line is a timestamp span, not link busy time. The second excludes the nominal last-packet spacing. The third includes that spacing.

An extra wait occurs after this finite group exactly when

$$g > g_m := 2D(2^m). \tag{5.3}$$

The first thresholds are $g_0 = 1$, $g_1 = 7/6$, and $g_2 = 533/420$. They increase to $2\ln 2$. Thus gains below the limiting threshold can still produce waiting during a finite initial part of the trace.

5.2 The three limiting regimes

Corollary 5.1 (Transition at $g_c = 2\ln 2$)**.** *For $1 \leq g < g_c$, additional ACK waiting ends after finitely many groups. For $g = g_c$, every group has a positive additional wait, but its length tends to zero. For $g > g_c$, the additional wait tends to the positive fraction $1 - g_c/g$ of an RTT.*

Proof. Use $D(2^m) \uparrow \ln 2$ in (4.2) and (5.2). If $g < g_c$, then $cD(2^m) > 1$ eventually, so $I_m = 0$ thereafter. Equations (4.5)–(4.6) exclude additional waiting inside each group. If $g \geq g_c$, then $cD(2^m) < 1$ for every finite m, so $B_m = m + 1$ and $I_m/R = 1 - cD(2^m)$, with the stated limits. □

Below the threshold, the inter-group duration tends to $2\ln 2/g > 1$ in RTT units. These index groups may span more than an RTT, but their separating gaps tend to zero. Thus they do not form persistent RTT-separated bursts. Indeed the eventual consecutive packet spacings are $1/[g(k+1)]$, repeated twice, and decrease to zero; a fixed RTT observation interval has first-to-last send span tending to R.

At and above the threshold, group m starts at $(m+1)R$ and ends before $(m+2)R$. Only in this regime can its span be identified directly with the span of the packets in that integer RTT. Equation (5.2) gives

$$\boxed{\lim_{m\to\infty} L_m/R = \frac{2\ln 2}{g}, \qquad \lim_{m\to\infty} I_m/R = 1 - \frac{2\ln 2}{g}.} \tag{5.4}$$

For $g = 2$ the limiting sending span is $\ln 2 \simeq 0.69315$ RTT; for $g = 4$ it is about 0.34657 RTT. A regular local pacing interval therefore does not prevent global concentration: the sender finishes its available packets earlier and then waits for feedback. The proof of this statement is the exact recurrence, not the heuristic integral $\int_W^{2W} 2\,dx/(gx)$.

6 Window growth in physical time

6.1 Below the threshold: exponent and amplitude

For $g < g_c$, the positive-part sum in (4.4) eventually stops changing. Define the finite sum

$$\delta_g = 1 - c + \sum_{i\geq 0} [1 - cD(2^i)]_+. \tag{6.1}$$

For every sufficiently large k, the equality $b_k = cH_k + \delta_g$ is exact. Both packets in batch k consequently have times $c\ln k + c\gamma_E + \delta_g + O(1/k)$. Inverting this relation changes the corresponding batch count by only $O(1)$, and there are two packets per batch. Therefore

$$N_g(\tau) = 2e^{(\tau-\delta_g)/c-\gamma_E} + O(1), \tag{6.2}$$

$$w_g(\tau) = A_g e^{g\tau/2} + O(1), \qquad A_g = 2e^{-(1+\delta_g)/c-\gamma_E} > 0. \tag{6.3}$$

The second line uses the exact one-RTT shift in (2.8).

For $g = 1$, $c = 2$ and $2D(2^i) \geq 1$, so $\delta_1 = -1$. In fact, for all $k \geq 1$,

$$b_k = 2H_k - 1, \qquad w_1(\tau) = 2e^{\tau/2-\gamma_E} + O(1). \tag{6.4}$$

The asymptotic multiplier per RTT is $\sqrt{e}$. It exceeds the Fibonacci multiplier ϕ, but is smaller than the immediate-send multiplier 2.

6.2 At and above the threshold: the within-RTT trajectory

When $g \geq g_c$, the exact solution reduces to

$$b_k = m + 1 + \frac{2}{g}(H_k - H_{2^m}), \qquad 2^m \leq k < 2^{m+1}. \tag{6.5}$$

For $n \geq 2$, ACK 2^{n-1} arrives at n because $a_{2^{n-1}} = b_{2^{n-2}} + 1 = n$; the next ACK arrives later. The case $n = 1$ follows from the initial packet. Thus every finite $g \geq g_c$ has the same integer-boundary window:

$$\boxed{w_g(n) = 1 + 2^{n-1} \quad (n \geq 1).} \tag{6.6}$$

The trajectories between those boundaries are different.

To see their shape, let $P_{n,g}(u)$ be the fraction of packets in $[n, n+1)$ that have been sent by $n+u$, $0 \le u < 1$; set $P_{n,g}(1) = 1$. For high gain the interval contains $2^n = 2q$ packets, where $q = 2^{n-1}$. Uniformly for $q \le k < 2q$,

$$b_k - n = \frac{2}{g}\ln(k/q) + O_g(1/q). \tag{6.7}$$

Hence the number of batches reached by phase u is $q\min\{e^{gu/2} - 1, 1\} + O_g(1)$. Counting two packets per batch gives

$$P_{n,g}(u) \longrightarrow \min\{e^{gu/2} - 1, 1\}. \tag{6.8}$$

The endpoint error affects at most $O_g(1)$ packets. The previous round's packets provide the present round's ACKs, so (2.8) yields

$$\boxed{w_g(n+u) = 2^{n-1}\min\{e^{gu/2}, 2\} + O_g(1), \quad 0 \le u < 1.} \tag{6.9}$$

For fixed finite g, the error can be bounded uniformly in u. At $g = g_c$, this simplifies to $w_g(\tau) = 2^{\tau-1} + O(1)$. Above the threshold, growth followed by a plateau persists within each RTT; one constant times 2^τ does not describe all phases with relative error tending to zero.

For comparison, (6.3) gives the lower-gain send distribution by taking differences of cumulative sends at the endpoints:

$$P_{n,g}(u) \longrightarrow \frac{e^{gu/2} - 1}{e^{g/2} - 1}, \qquad 1 \le g < g_c. \tag{6.10}$$

The two limits agree at g_c. They describe the fraction of this RTT's packets sent, not the normalized window. Figure 1 plots the high-gain expression directly.

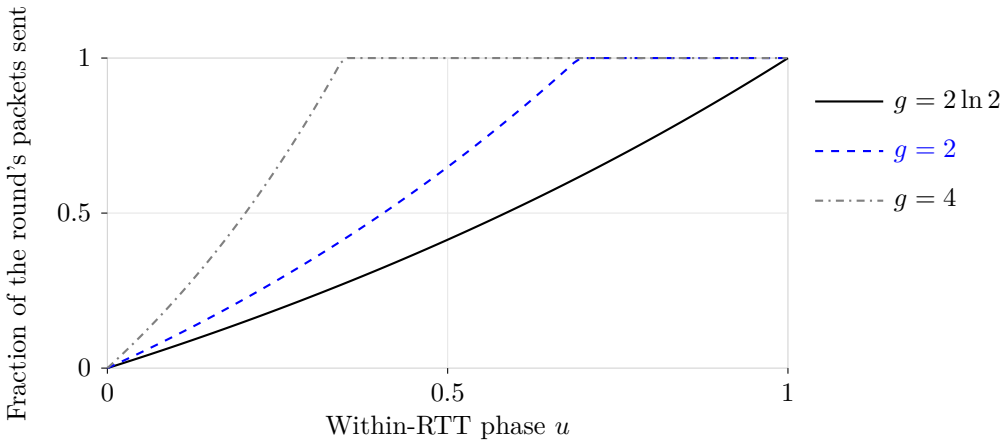


Figure 1: Limiting send-time distributions from (6.8), drawn from the formula, not experimental traces. Increasing gain compresses the sending portion of an RTT while preserving positive local packet spacing at every finite window and gain.

Corollary 6.1 (Growth exponent and one-RTT multiplier)**.** *For the specified scheduler and any fixed $g \ge 1$,*

$$\boxed{\begin{aligned} \lim_{\tau\to\infty} \frac{\ln w_g(\tau)}{\tau} &= \min\{g/2, \ln 2\}, \\ \lim_{\tau\to\infty} \frac{w_g(\tau+1)}{w_g(\tau)} &= \min\{e^{g/2}, 2\}. \end{aligned}} \tag{6.11}$$

Proof. Below the threshold apply (6.3). At and above it apply (6.9): the coefficient is bounded between 1/2 and 1, and advancing exactly one RTT preserves phase and doubles its leading term. The uniform bounded remainder vanishes relative to that term. □

The physical-time growth exponent is the first limit divided by R. All slow-start formulas describe an indefinitely continued growth phase. With finite ssthresh they apply only up to the transition, not to the later connection as t tends to infinity.

7 Gain monotonicity and the burst limit

For fixed packet index, increasing g never delays its send. To prove this, compare $g_2 \geq g_1$ using (2.2)–(2.4). Initially both send packet 0 at zero and $T_0(g_2) \leq T_0(g_1)$. If preceding corresponding sends and queue tails are no later at g_2, the next corresponding ACK is also no later. The maximum defining b_k is no larger, and the two new spacings are no larger. Induction proves the claim for sends and tails. Equation (2.8) then gives $w_{g_2}(\tau) \geq w_{g_1}(\tau)$. The monotonicity need not be strict, as (6.6) demonstrates.

For $g \geq g_c$, every packet j in group m has $s_j(g) = m + 1 + d_j/g$, with $d_j \geq 0$ independent of gain. These packets have $2^{m+1} - 1 \leq j \leq 2^{m+2} - 2$. Consequently, including $j = 0$,

$$\boxed{\lim_{g\to\infty} s_j(g) = \lfloor \log_2(j+1) \rfloor.} \tag{7.1}$$

This is precisely the immediate-send trace: one packet at zero, two at one, four at two, and so on. Local pacing and global concentration coexist for finite gain; only the limit collapses each group's spacing to zero.

There is a boundary subtlety. For finite high gain, $w_g(n) = 1 + 2^{n-1}$, whereas the burst sender has $w_{\text{burst}}(n) = 2^n$. There is no contradiction: ACKs that arrive just after n for finite gain coalesce at n in the limit. Counting at a jump is not continuous under convergence of event times. Equation (7.1) is a packet-time limit, not pointwise convergence of the window at every burst jump.

8 Extension to loss-free congestion avoidance

8.1 A continuous event history across the threshold

Take an integer threshold $\theta \geq 2$ and choose the phase by the window before the ACK:

$$w_k = \begin{cases} w_{k-1} + 1, & w_{k-1} < \theta, \\ w_{k-1} + 1/w_{k-1}, & w_{k-1} \geq \theta. \end{cases} \tag{8.1}$$

The CA expression is the real-valued full-packet form of RFC 5681, Section 3.1, equation (3), not every possible Reno byte-counting implementation [1]. For example, at $\theta = 10$, ACK 9 raises the window to 10 using SS, and ACK 10 raises it to 10.1 using CA. Flight, the existing reservations, and the queue clock are retained at the transition.

For a real-valued window, use every available whole-packet reservation: $F + Q = \lfloor w \rfloor$. An ACK reduces the previous count by one, so the number of new packets is exactly

$$\boxed{m_k = 1 + \lfloor w_k \rfloor - \lfloor w_{k-1} \rfloor.} \tag{8.2}$$

In CA this is one or two: the replacement packet and, when an integer window threshold is crossed, one additional packet. Let J_k be the total number of packets ever reserved after ACK

k, including the initial one. Then $J_k = k + \lfloor w_k \rfloor$, and the full recurrence is

$$a_k = s_{k-1} + 1, \qquad b_k = \max\{a_k, T_{k-1}\}, \tag{8.3}$$

$$s_{J_{k-1}+j} = b_k + \frac{j}{gw_k} \quad (0 \le j < m_k), \qquad T_k = b_k + \frac{m_k}{gw_k}. \tag{8.4}$$

It defines the whole SS-to-CA trace without clearing outstanding packets. With $K(\tau) = \#\{k : a_k \le \tau\}$, the exact observed window is $\mathrm{cwnd}(t) = Mw_{K(t/R)}$.

8.2 Window growth indexed by ACK count

At CA entry use a local index j and initial window $u_0 = a \ge 1$. Squaring $u_j = u_{j-1} + 1/u_{j-1}$ gives

$$\boxed{u_j^2 = a^2 + 2j + \sum_{i=0}^{j-1} \frac{1}{u_i^2}.} \tag{8.5}$$

Since $u_i^2 \ge a^2 + 2i$, the sum is $O(\ln(j+1))$, and $u_j^2 = a^2 + 2j + O(\ln(j+1))$. This is an ACK-count result. Replacing j with t/R would lose the many ACKs that can arrive within one RTT.

8.3 A proved logarithmic term in the queue clock

Let k_0 be the last SS update and define, over subsequent CA ACKs,

$$\ell_k = [a_k - T_{k-1}]_+, \qquad \mathcal{I}_K = \sum_{k=k_0+1}^{K} \ell_k. \tag{8.6}$$

The calligraphic $\mathcal{I}_K$ is cumulative waiting; it differs from the single slow-start group wait I_m of Section 5. Telescoping (8.4) and substituting (8.2) gives the exact identity

$$T_K - T_{k_0} = \mathcal{I}_K + \frac{1}{g} \sum_{k=k_0+1}^{K} \frac{1}{w_k} + \frac{1}{g} \sum_{k=k_0+1}^{K} \frac{\lfloor w_k \rfloor - \lfloor w_{k-1} \rfloor}{w_k}. \tag{8.7}$$

The sums account respectively for replacing acknowledged packets and for additional packets made possible by whole-MSS window growth.

Theorem 8.1 (CA queue-clock asymptotic)**.** *For this scheduler, fixed $g \ge 1$, and an indefinitely continued loss-free CA phase,*

$$\boxed{T_K = \mathcal{I}_K + \frac{1}{g}(w_K + \ln w_K) + C + O(1/w_K),} \tag{8.8}$$

where C depends on the finite initial history and fixed parameters.

Proof. The update gives the exact identity

$$\frac{1}{w_k} = (w_k - w_{k-1}) - \frac{1}{w_{k-1}^2 w_k}. \tag{8.9}$$

By (8.5), $w_k = \Theta(\sqrt{k})$ after the finite initial history. The correction series therefore converges, with tail $O(K^{-1/2}) = O(1/w_K)$. The first sum in (8.7) is $w_K + C_1 + O(1/w_K)$.

For the second sum, each sufficiently large integer threshold n is crossed exactly once. If v_n is the first window at or above n, its preceding value x obeys $n - 1 < x < n$, and

$$n \le v_n = x + 1/x < n + \frac{1}{n-1}. \tag{8.10}$$

Hence $v_n = n + O(1/n)$ and $1/v_n = 1/n + O(1/n^3)$. The extra-packet sum is $\ln w_K + C_2 + O(1/w_K)$: the reciprocal-error series converges, and replacing $\ln\lfloor w_K \rfloor$ with $\ln w_K$ changes only the stated remainder. Substitution proves (8.8). □

The logarithm here comes from additional whole packets at successive integer window thresholds. Equation (8.8) is a queue-clock theorem, not yet a closed asymptotic for cwnd(t). One still needs the growth of $\mathcal{I}_K$ and the relation between the queue tail and observation time. For instance, at $g = 1$ the two further estimates

$$\mathcal{I}_K = \tfrac{1}{2} \ln w_K + O(1), \qquad T_{K(\tau)} - \tau = O(1) \tag{8.11}$$

would imply $\tau = w + \frac{3}{2} \ln w + O(1)$ and thus $w(\tau) = \tau - \frac{3}{2} \ln \tau + O(1)$. Those estimates are not proved here; the coefficient $3/2$ is an open candidate, not a theorem of this paper. The CA model has an increasing window and is not a statistical steady state.

9 Relation to implementations and earlier work

Pacing-related timing and performance effects are established topics. Aggarwal, Savage, and Anderson analyze pacing's benefits and failures, including feedback and synchronization effects and loss-free short-flow latency caused by spreading transmissions [2]. The premise that pacing can delay feedback is therefore not itself a novelty claim. The specific result here is an exact solution and comparison proof for the declared immutable-reservation scheduler.

RFC 9002, Section 7.7, gives pacing-rate and interval examples with a gain above one and explicitly permits different pacer architectures [3]. The pacing Internet-Draft describes implementation choices and their consequences [4]. These describe a broader design space than the recurrence solved here. A nominal rate gW/R does not specify whether old deadlines are changed, whether service is reserved at admission or transmission, or how idle intervals and batching are handled.

As a concrete versioned example, Linux v6.12's generic TCP pacing path uses a slow-start ratio when cwnd $<$ $\lfloor$ssthresh$/2\rfloor$ (both values in segments) and the other ratio otherwise; the documented defaults are 200% and 120% [5]. The threshold $2 \ln 2$ proved here neither derives those engineering defaults nor proves that a smaller value is sufficient in Linux. The actual SCP revision that motivated this study also differs: its `scp_output` returns while an older pacing batch remains, and its timer can release several due packets together. It is not the per-ACK append scheduler of Section 2 [6].

A continuously repriced fluid model would differ as well. If its active send rate were $x(\tau) = gw(\tau)$, fixed-delay slow-start feedback would give $w'(\tau) = gw(\tau - 1)$. An exponential mode would satisfy $ze^z = g$, not $z = g/2$. This follows directly by substitution. It is not a conflicting solution of our packet recurrence: the fluid equation assumes immediate realization of the current rate, whereas our old reservations keep their intervals and timestamps. A flight-feasibility analysis would also be needed before treating that fluid mode as an admissible sender. Results from these different models should not be combined as if they described one scheduler.

The limits of the present results are consequently specific. Finite initial windows other than one, delayed ACKs, variable RTT, finite link capacity, loss recovery, and mutable reservations can change the trace. The model has no network penalty for a large gain, so it cannot identify an optimal gain balancing delay, loss, fairness, and throughput. Equal growth exponents also do not establish equal finite-transfer completion times.

10 Conclusion

For an explicitly defined per-ACK append pacer, the transmission times admit an exact harmonic-sum solution for every fixed gain $g \geq 1$. The solution establishes the Fibonacci comparison bound, the transition from eventual continuous pacing to persistent RTT-separated sending intervals, and the burst limit of individual packet timestamps. It also determines the growth exponent and the within-RTT window trajectory, showing why those two notions of

growth must be distinguished. In congestion avoidance, exact packet accounting isolates a logarithmic queue-clock term, while the accumulated ACK-waiting term remains the missing part of a full physical-time asymptotic.

A Proof of the Fibonacci lower comparison

This appendix completes Theorem 3.1 without assuming that earlier reservations necessarily produce earlier sends.

At $g = 1$, equation (6.4) gives

$$s_0 = 0, \qquad s_{2k-1} = 2H_k - 1, \qquad s_{2k} = 2H_k - 1 + \frac{1}{k+1}. \tag{A.1}$$

Its consecutive spacings are

$$s_{j+1} - s_j = \frac{1}{\lfloor (j+1)/2 \rfloor + 1} \quad (j \geq 0). \tag{A.2}$$

For fixed-batch round $n \geq 1$, put $p = f_{n+1}$, $q = f_{n+2}$. Its first packet index is $J_n = p + q - 2$, and its send times are

$$\widehat{s}_j = n + \frac{j - J_n}{q}, \qquad J_n \leq j < J_n + q. \tag{A.3}$$

Within this range the consecutive differences of $s_j - \widehat{s}_j$ are the above decreasing spacings minus $1/q$. They are positive before $j = 2q - 3$, zero for $j = 2q - 3, 2q - 2$, and negative afterwards. A maximum is therefore attained at $j_* = 2q - 3$. This index is in the batch because $j_* - J_n = q - p - 1 \geq 0$ and $J_n + q - 1 - j_* = p \geq 1$.

Substitute j_* into the two send formulas to obtain

$$\begin{aligned} \max_{J_n \leq j < J_n + q} (s_j - \widehat{s}_j) &= 2H_q - n - 2 + \frac{p-1}{q} = -E_n, \\ E_n &= n + 2 - \frac{p-1}{q} - 2H_q. \end{aligned} \tag{A.4}$$

Initially $p = 1$, $q = 2$, and $E_1 = 0$. In the next batch the pair becomes $(q, p + q)$. Direct subtraction gives

$$E_{n+1} - E_n = 1 - \frac{q-1}{p+q} + \frac{p-1}{q} - 2\sum_{i=1}^{p} \frac{1}{q+i}. \tag{A.5}$$

Use the elementary identity

$$\frac{q+p-i}{q(q+p)} - \frac{1}{q+i} = \frac{i(p-i)}{q(q+p)(q+i)} \geq 0 \quad (1 \leq i \leq p). \tag{A.6}$$

The sum of twice the first terms on the left is $(p-1)/q + (p+1)/(p+q)$. Consequently,

$$E_{n+1} - E_n = 2\sum_{i=1}^{p} \frac{i(p-i)}{q(q+p)(q+i)} \geq 0. \tag{A.7}$$

Thus $E_n \geq 0$ for all n, and $s_j \leq \widehat{s}_j$ for every packet in every batch. Packet 0 is common to both senders. Actual transmission counts, and then ACK counts by the common delay, are therefore ordered. Equation (2.8) proves $w_1(\tau) \geq w_{\text{fixed}}(\tau)$ at every time. Section 7 extends this to $g \geq 1$, and the fixed-batch boundary values (3.4) finish the proof.

References

[1] M. Allman, V. Paxson, and E. Blanton. *TCP Congestion Control*. RFC 5681, September 2009. https://www.rfc-editor.org/rfc/rfc5681.

[2] A. Aggarwal, S. Savage, and T. Anderson. Understanding the Performance of TCP Pacing. *IEEE INFOCOM*, pp. 1157–1165, 2000. https://cseweb.ucsd.edu/~savage/papers/Infocom2000pacing.pdf.

[3] J. Iyengar and I. Swett. *QUIC Loss Detection and Congestion Control*. RFC 9002, May 2021. https://www.rfc-editor.org/rfc/rfc9002.

[4] M. Welzl, W. Eddy, V. Goel, and M. Tüxen. *Pacing in Transport Protocols*. `draft-irtf-iccrg-pacing-02`, June 26, 2026. Work in progress; this citation fixes the version discussed. https://datatracker.ietf.org/doc/html/draft-irtf-iccrg-pacing-02.

[5] Linux v6.12. `tcp_update_pacing_rate` and TCP pacing ratio documentation. Versioned source; parameter documentation.

[6] Skai Uijing. SCP source, `scp.c`, revision `bf9d8873b0f8`.